

report_item

1 1 4 2 0 2

report_item.sh – § . . reporting- directive engine

Field	Value
Revision	1
Created	2026-07-15
Last modified	2026-07-15T00:00:00Z
Status	active
Script	constitution/scripts/reporting/report_item.sh
Authority	constitution/Constitution.md §11.4.202

Overview

`report_item.sh` turns a plain-language REPORT — delivered by the §11.4.140 reporting directives `ISSUE` / `BUG` / `TASK` , or by any other caller — into a **real, fully-populated, fully-synced workable item**.

```

report text
  └─(1) CREATE → the workable-items SQLite SSoT (§11.4.93 / §11.4.95)
    |
    |           Type (§11.4.16) + Status=Queued (§11.4.15) + stable id
    |           (§11.4.54)
    |           + comprehensive structured description (§11.4.148 /
    |           §11.4.171)
    └─(2) SYNC  → every derived document regenerated FROM the DB
      |
      |           (Issues / Fixed / summaries + HTML/PDF/DOCX siblings)
      └─(3) PUSH → every configured external tracker (§11.4.148 D5)
        |
        |           absent credentials / client ⇒ honest SKIP, NEVER a
        |           faked push

```

A report that is discussed but never tracked is a §11.4 PASS-bluff at the requirements-intake layer (§11.4.197 — a started requirement never evaporates).

Prerequisites

- `python3` (+PyYAML) — reads the consumer config.
- The `workable-items` Go binary (committed in the constitution, or built, or pointed at via `$WORKABLE_ITEMS_BIN`).
- A **consumer-owned config** (the engine refuses to guess — §11.4.6):

```

mkdir -p .helix
cp constitution/scripts/reporting/reporting.example.yaml .helix/reporting.yaml
$EDITOR .helix/reporting.yaml # db, id_prefix, sync_command, trackers

```

Search order: `$HELIX_REPORTING_CONFIG` → `.helix/reporting.yaml` → `config/reporting/reporting.yaml`.

Usage examples

A defect (Type=Bug, fixed):

```
bash constitution/scripts/reporting/report_item.sh --kind bug \  
--report 'Audio drops to stereo on the AVR after a reboot; the receiver shows PCM 2.0.'
```

An internal workstream item (Type=Task, fixed):

```
bash constitution/scripts/reporting/report_item.sh --kind task \  
--report 'Extend the flasher to take an explicit --serial (§11.4.200).'
```

A generic report — the type MUST be classified into {Bug|Feature|Task}:

```
bash constitution/scripts/reporting/report_item.sh --kind issue --type Feature \  
--title 'Per-display font scale' --report 'The TV needs its own default font size.'
```

Rich, structured intake (omitted sections ⇒ honest UNKNOWN:, never invented)

```
bash constitution/scripts/reporting/report_item.sh --kind bug \  
--title 'Subtitles render the control-menu label' \  
--report 'The 2nd display shows the chrome label instead of the dialogue cue.' \  
--scope 'presenter: PresenterAccessibilityService.kt' \  
--repro 'Play a RU title, open Audio&Subtitles, press back.' \  
--acceptance 'Only dialogue cues render; the denylist rejects chrome labels.' \  
--severity Critical --json
```

Preview without writing anything:

```
bash constitution/scripts/reporting/report_item.sh --kind bug --report '...' --dry-run
```

Flag	Meaning
<code>--kind bug task issue</code>	required. <code>bug</code> $\Rightarrow$ Type=Bug, <code>task</code> $\Rightarrow$ Type=Task, <code>issue</code> $\Rightarrow$ needs <code>--type</code>
<code>--type Bug Feature Task</code>	required for <code>--kind issue</code> (§11.4.16 closed set)
<code>--autonomous</code>	for <code>--kind issue</code> with no <code>--type</code> : use the §11.4.16 lowest-stakes default <code>Task</code> and record the defaulted classification in the item
<code>--report /</code> <code>--report-file /</code> <code>stdin</code>	the report text (in that precedence)
<code>--title</code>	optional; derived from the report's first sentence when omitted
<code>--scope</code> , <code>--repro</code> , <code>--acceptance</code>	optional structured sections
<code>--severity</code> , <code>--reported-by</code>	optional; fall back to the config defaults
<code>--no-sync</code> , <code>--no-tracker</code>	skip step 2 / step 3
<code>--dry-run</code> , <code>--json</code>	preview / machine-readable result

Edge cases (all honest, none faked)

Situation	Behaviour
No config found	Exit 2 with the template path. The engine never guesses your DB path (§11.4.6).
<code>--kind issue</code> without <code>--type</code>	Refused (exit 64). Classify it, ASK the operator (§11.4.66), or pass <code>--autonomous</code> .
Tracker credentials unset	Verdict <code>SKIP</code> , reason <code>credentials_absent: unset env: NAME ...</code> — variable names only, never values (§11.4.10).
Tracker client not built	Verdict <code>SKIP</code> , reason <code>tracker_client_absent</code> (PENDING-OPERATOR-INPUT).
Tracker command fails	Verdict <code>FAIL</code> with its exit code — reported, never hidden. The item still exists.
Sync command fails	Exit 5 . The item IS created (the DB is the SSoT); re-run the sync.
No <code>sync_command</code> configured	Honest <code>SKIPPED</code> with reason — never an invented command.

A tracker `PASS` is recorded **only** after a real push command exits 0.

Internal behaviour

1. **Config** → python3/PyYAML → shell-safe `KEY=value` pairs (values quoted).
2. **Binary** → `$WORKABLE_ITEMS_BIN` → config → committed constitution binary → `go build` .
3. **Description** → the §11.4.148 D2 template. Sections the report does not state are written as explicit `UNKNOWN:` gaps — never invented (§11.4.6).
4. **Create** → `workable-items add <Type> <Severity> ...` ; the assigned id is parsed from the binary's own output.
5. **Sync** → the consumer's `sync_command` (`{db}` substituted), run from the project root.

6. **Trackers** → for each: presence-check `required_env` (never value-read),
render `{db} / {id}` , run, capture.
7. **Evidence** → `<evidence_dir>/{id}<ts>/result.json` + `create.log` +
`sync.log` + `tracker_<name>.log` (§11.4.5 / §11.4.69).

Decoupling (\$. . / \$. .)

The engine carries **zero project literals** and is inherited **by reference** (never copied). Every project-specific value is consumer-owned DATA. The `CM-REPORTING-DIRECTIVES` gate enforces this (a hardcoded track path or id prefix in the engine → gate FAILs).

Related

Path	Purpose
<code>constitution/actions/registry.yaml</code>	the <code>ISSUE / BUG / TASK</code> rows + the grammar
<code>constitution/scripts/constitution_prefix_lib.sh</code>	the §11.4.140 parser (all six forms)
<code>constitution/scripts/reporting/reporting.example.yaml</code>	the consumer config template
<code>constitution/scripts/gates/cm_reporting_directives.sh</code>	the pre-build gate (+ its §1.1 mutation test)
<code>tests/reporting/test_report_item_e2e.sh</code>	the real-DB end-to-end + self-validation suite
<code>constitution/scripts/workable-items/</code>	the DB SSoT binary (§11.4.93)

Last verified: 2026-07-15 — end-to-end against a real SQLite DB: item created with the correct Type/Status/id, docs regenerated from the DB, both trackers SKIPped with honest reasons (38/38 PASS); gate 21/21 PASS with all 6 paired mutations caught.